

Abbreviated Lambdas for Fun and Profit

Contents

- [Revisions](#)
- [Motivation](#)
- [Proposal](#)
- [SFINAE and noexcept](#)
- [Examples](#)
- [Hyper-abbreviated lambdas](#)
- [Prior Work and Effects on Existing Code](#)
- [Acknowledgements and References](#)

Revisions

Since r0, this paper focused on a single syntax for abbreviating lambdas, and includes a stronger motivation for the need for SFINAE and noexcept. The section on abbreviated forwarding has been pulled out into its own paper, [Forward without forward](#), and a new section has been introduced discussing expression-based lambdas.

Motivation

The introduction of lambdas in C++11 transformed the landscape of C++ tremendously by allowing users to write arbitrary functions, predicates, and callbacks in-situ. Suddenly, instead of having to write function objects somewhere far from the code, we could write them inline. C++14 improved lambdas both by expanding their functionality (with generic lambdas) and by making them shorter (by being able to use `auto` instead of spelling out the full type name of the parameters).

While lambdas are at the point where nearly arbitrary functionality is possible, it is still cumbersome and at times unnecessarily verbose to write out lambdas in the simplest of cases. A disproportionate number of lambda expressions are extremely short, consisting solely of a single expression in the body. These simple lambdas typically occur in one (or more) of these situations:

- Unary/Binary predicates into algorithms.
- Callbacks
- Binding - simply reordering or adding new arguments for other functions. Basically, better `std::bind`.
- Lifting - taking overloaded functions and lifting them into a generic lambda so that they can be passed into a function.

In all these cases, the amount of code necessary to write a lambda seems excessive compared to the actual functionality being expressed. The lifting of overload sets [1] is a particularly awkward situation. Since we cannot simply write:

```
template <class T>
T twice(T x) { return x + x; }

template <class I>
void f(I first, I last) {
    transform(first, last, twice); // error
}
```

we have to instead write, at the very least:

```
transform(first, last, [](auto&& x) -> decltype(auto) {
    return twice(std::forward<decltype(x)>(x));
});
```

Very little of that code adds value to the reader.

Lambdas are unique in C++ in that they often are meaningless in a vacuum - they rely on the immediate context that comes from the expressions that contain them and code that precedes them. Since they often appear as sub-expressions, there is intrinsic value in making them shorter - if we can make them shorter by simply removing aspects of them that do not provide meaning to other readers of our code. In single-expression lambdas, these include:

- `return` for returning. The lambdas this proposal refers to are single-expression. Can't we just assume that we're returning?
- `auto&&` for arguments. With C++14, this has become the near-default usage for lambdas. It's almost never wrong. Even if the lambda isn't intended to be used polymorphically, it's preferable to use `auto&&` rather than writing out the full name of the type. But once we're always writing `auto&&`, it itself doesn't actually have any meaning (if it ever did). Consider a predicate for sorting `lib::Widgets` by id:

```
// C++11
[](lib::Widget const& a, lib::Widget const& b) { return a.id() < b.id(); }

// C++14 - Widget is simply inferred
[](auto&& a, auto&& b) { return a.id() < b.id(); }
```

It's a full third shorter to just use `auto&&`, but do we even need that?

- Capture. Typically, this sort of lambda is invoked immediately. It will not be stored and will not last past the full-expression containing it. Similar to how `auto&&` is the de-facto parameter of choice, `{&}` is the de-facto capture of choice.
- Trailing-return type and `noexcept`. In the `twice()` example earlier, I used `decltype(auto)`. This isn't important for predicates (they should just be returning `bool` anyway), but is very important for binding and lifting. If the underlying function we're wrapping returns a reference type, we want to maintain that. But even this isn't quite sufficient - we want the lambda to be as light a wrapper as possible, and using `decltype(auto)` hides some of the functionality from metaprogramming contexts. To avoid hiding the functionality, users have to write the function body in triplicate. That is, to really completely lift the `twice()` function to a lambda, you have to write:

```
[](auto&&... args) noexcept(noexcept(twice(std::forward<decltype(args)>(args)...)) -> decltype(twice(std::forward<decltype(args)>(args)...)) { return twice(std::forward<decltype(args)>(args)...);
```

This lambda would be transparent to simply having used `twice()` by name. But nobody wants to write this ever, this is an unmaintainable monstrosity. The importance of maintaining SFINAE-friendliness and `noexcept` will be discussed later.

This proposal seeks to eliminate all of unnecessary boilerplate in simple lambdas, without preventing users from writing the arbitrarily complex lambdas that we can today.

Proposal

This paper proposes the creation of a new lambda introducer, `=>`, which allows for a single expression in the body that will be its return statement. The usage of `=>` will address all of the bullet points above.

First, use of `=>` allows for (and requires) omitting the `return` keyword. The programmer simply provides an expression:

```
[](auto&& a, auto&& b) { return a.id() < b.id(); } // C++14
[](auto&& a, auto&& b) => a.id() < b.id() // this proposal
```

Second, use of `=>` will allow for omitting types in the parameter list, defaulting to `auto&&`. This flips the treatment of a single identifier from identifying an unnamed argument of that type to identifying a forwarding reference with that name:

```
[](x, y) { ... } // lambda taking two unnamed parameters of types x and y
[](x, y) => (x > y) // lambda taking two forwarding references named x and y
```

```
// this is equivalent to std::greater<>()
```

Or to further the prior example:

```
[](auto&& a, auto&& b) { return a.id() < b.id(); } // C++14
[](a, b) => a.id() < b.id() // this proposal
```

While types *may be* omitted, it is not mandatory.

Third, use of `=>` will allow for omitting the capture for the lambda, defaulting to `{&}`. This allows for a very natural reading of the lambda's body, as it looks the same as any other code you might write which would naturally have access to all the other names in its scope. Omitting the capture would also allow for omitting parentheses around single-argument lambdas:

```
[&](auto&& w) { return w.id() == id; } // C++14
w => w.id() == id // this proposal
```

While capture *may be* omitted, it is not mandatory, and capture may continue to be used as normal.

Lastly, use of `=>` will synthesize a SFINAE-friendly, `noexcept`-correct lambda. That is, the now very concise

```
x => test(x)
```

shall be exactly equivalent to the C++14 lambda:

```
[](auto&& x) noexcept(noexcept(test(x))) -> decltype(test(x)) { return test(x); }
```

SFINAE and noexcept

The motivation for lambdas is often to transparently wrap an expression and make it usable as a function. If I have an overload set, like:

```
int& foo(int& ) noexcept;
std::string foo(char const* );
```

I can use `foo` in generic code directly by name and query whether it's invocable with a given argument type and query whether it's `noexcept` with a given argument type. This is *prima facie* important and useful. If I naively wrap `foo` in a lambda that simply returns:

```
[](auto&& arg) -> decltype(auto) { return foo(arg); }
```

I immediately lose both abilities. This lambda is *never* `noexcept` and advertises itself as being invocable with any argument type. Since the user took the effort to deliberately mark `foo(int& )` as being `noexcept`, it seems wasteful to just drop it on the floor.

Similarly, given that we only have a single expression, there is no advantage to writing `decltype(auto)` over `decltype(foo(arg))`. The latter is strictly more expressive, and more useful in more contexts. In contexts where SFINAE doesn't matter, it doesn't matter - the two are equivalent. But in contexts where it does:

```
bool call_me_maybe(function<bool(int)> ); // #1
bool call_me_maybe(function<bool(string) > ); // #2

// non-SFINAE-friendly lambda
call_me_maybe([](auto x) { return x == 2; }); // error! lots and lots of diagnostic

// SFINAE-friendly C++14 lambda
call_me_maybe([](auto x) -> decltype(x == 2) { return x == 2; }); // OK, calls #1

// this proposal, single-expression lambdas with => are always SFINAE-friendly
// no need to copy the whole body of the function
call_me_maybe(x => x == 2); // OK, calls #1
```

While having the lambdas in this case have a return type of `auto` and no `noexcept`-specification is more consistent with the language as it stands, having the lambdas, especially these kinds of lambdas, have a return type of `decltype(expression)` and `noexcept`-specification of `noexcept(noexcept(expression))` saves the user from typing a bunch of repetitive boilerplate, and gets us to the point where we can just write *simple* code that just does the Right Thing™.

Examples

Transparently binding an overloaded member functions `func` to an instance `obj`:

```
// C++14
[&](auto&&... args) noexcept(noexcept(obj.func(std::forward<decltype(args)>(args)...))) -> decltype(obj.func(std::forward<decltype(args)>(args)...)) { return obj.func(std::forward<decltype(args)>(args)...); }

// this proposal
(args...) => obj.func(std::forward<decltype(args)>(args)...)
```

Sorting in decreasing order. Currently, we would rely on one of the many *named* function objects in the standard library to do this for us. But that involves having to actually remember what these things are named. Why do I need to know about and remember `std::greater<>` if I already know about `>`? The gain here isn't so much in the length (only two characters), but in the readability.

```
std::sort(v.begin(), v.end(), std::greater<>()); // C++14
std::sort(v.begin(), v.end(), (x,y) => x > y); // this proposal
```

Once we move from directly working on the elements to working on other functions of the elements, the gain becomes much bigger. Sorting in decreasing order by ID is both shorter and much more readable:

```
std::sort(v.begin(), v.end(), [](auto&& x, auto&& y) { return x.id() > y.id(); }); // C++14
std::sort(v.begin(), v.end(), std::greater<>(), &lib::Widget::id); // Ranges TS, with projection
std::sort(v.begin(), v.end(), (x,y) => x.id() > y.id()); // this proposal
```

Check if all of the elements have some predicate satisfied - by element directly:

```
std::all_of(v.begin(), v.end(), [](auto&& elem) { return elem.done(); }); // C++14, direct
std::all_of(v.begin(), v.end(), v.end(), std::mem_fn(&lib::Element::done)); // C++14, with pointers to member functions
std::all_of(v.begin(), v.end(), e => e.done()); // this proposal
```

or on an external object:

```
std::all_of(v.begin(), v.end(), [&](auto&& elem) { return obj.satisfied(elem); }); // C++14, directly
std::all_of(v.begin(), v.end(), v.end(), id, &lib::Widget::id); // Ranges TS, with projection
std::all_of(v.begin(), v.end(), elem => obj.satisfied(elem)); // this proposal

// * as long as Element::satisfied() isn't overloaded or a template
```

Looking for an element by id:

```
auto it = std::find_if(v.begin(), v.end(), [&](auto&& elem) { return elem.id() == id; }); // C++14
auto it = std::find(v.begin(), v.end(), id, &lib::Widget::id); // Ranges TS, with projection
auto it = std::find_if(v.begin(), v.end(), elem => elem.id() == id); // this proposal
```

Number of pairwise matches between two containers, using `std::inner_product` with two callables. With this proposal, it's negligibly shorter, but importantly you don't have to know the names of things - you just write the expressions you want to write:

```
int matches = std::inner_product(as.begin(), as.end(), bs.begin(), 0, // C++14, example from cppreference.com
std::plus<>(), std::equal_to<>());
int matches = std::inner_product(as.begin(), as.end(), bs.begin(), 0, // this proposal
(a,b) => a + b, (a,b) => a == b);
```

In all of these cases, the lambda is really simple. Which is the point. Let's make it simpler to write simpler things.

Hyper-abbreviated lambdas

This proposal is about as abbreviated as you can get, without loss of clarity or functionality. But we can always go deeper. Any proposal on abbreviating lambdas would be incomplete without mentioning that numerous languages (including, but not limited to Swift, Elixir, and Scala), as well as the Boost.Lambda library, allow for writing expressions that refer to arguments by number - which would then synthesize a new closure. An example using Swift's syntax with the sorting by id predicate that I have been using

throughout:

```
[](auto&& a, auto&& b) { return a.id() < b.id(); } // C++14 (51 characters)
(a, b) => a.id() < b.id() // this proposal (26)
$0.id() < $1.id() // even more abbreviation (18)
```

or checking if an element satsifes an object's test:

```
[&](auto&& elem) { return obj.satisfies(elem); }; // C++14 (50 characters)
elem => obj.satisfies(elem); // this propsoal (29)
obj.satisfies($0); // even more abbreviation (19)
```

While \$ directly is probably not the best choice in C++, there are other chracters currently unusable in this context whose meaning we could overload here (e.g. &). In my opinion, this next step in abbreviation is unnecessary as this proposal gets you most of the way there and now we start moving towards a brevity that sacrifices some clarity.

Prior Work and Effects on Existing Code

The original paper introducing what are now generic lambdas [2] also proposed extensions for omitting the *type-specifier* and dropping the body of a lambda if it's a single expression. This paper provides a different path towards those that same goal.

The usage of => (or the similar ->) in the context of lambdas appears in many, many programming languages of all varieties. A non-exhaustive sampling: C#, D, Erlang, F#, Haskell, Java, JavaScript, ML, OCaml, Swift. The widespread use is strongly suggestive that the syntax is easy to read and quite useful.

The token => can appear in code in rare cases, such as in the context of passing a the address of the assignment operator as a template non-template parameter, as in x. However, such usage is incredibly rare, so this proposal would have very limited effect on existing code. Thanks to Richard Smith for doing a search.

Acknowledgements and References

Thanks to Andrew Sutton and Tomasz Kaminski for considering and rejecting several bad iterations of this proposal. Thanks to Richard Smith for looking into the practicality of this design. Thanks to Nicol Bolas for refocusing the paper. Thanks to John Shaw for putting up with many crazy ideas.

Thanks especially to Adam Martin for presenting this proposal at Kona, and Nathan Myers for valuable feedback.

[1] [Overload sets as function arguments](#)

[2] [Proposal for Generic \(Polymorphic\) Lambda Expressions](#)